Pharmacy-Sales-Predictor

OTC Medicine Demand Forecasting

Submitted by

Kushagra Nigam

Registration No: 25BAI11055

Course: CSA2001 — Fundamentals in AI and ML

March 2026

1. Title

Pharmacy-sales-predictor: AI-Driven OTC Medicine Demand Forecasting

2. Introduction

The pharmaceutical retail sector relies heavily on accurate inventory management. A pharmacist who stocks too little risks patient dissatisfaction and lost revenue; one who stocks too much risks expired product waste and capital lockdown. Traditional inventory approaches depend on intuition and historical averages, both of which fail to account for temporal patterns, regional variation, and seasonal disease cycles.

Pharmacy-sales-predictor is a command-line decision support tool that applies supervised machine learning to historical OTC sales data. It predicts which medicine category will experience the highest demand for a given country and date, enabling pharmacists to make data-driven restocking decisions before the week begins.

The project was implemented entirely in Python and runs via the terminal, with no graphical interface required. Two classification models were trained and compared: a Random Forest classifier as the baseline and a Gradient Boosting classifier as the optimised solution.

3. Motivation of the Project

The motivation for Pharmacy-sales-predictor stems from a common observation in community pharmacies: stock decisions are reactive rather than predictive. Pharmacies reorder medicine after noticing low stock rather than anticipating demand in advance.

This problem is compounded by the fact that OTC medicine demand is not random — it is strongly influenced by season (cold medicine demand rises in winter), geography (regional disease prevalence varies), and day-of-week patterns (weekend demand often differs from weekday). None of these signals are captured in a simple reorder-when-empty system.

The availability of sales data and accessible machine learning libraries in Python made this an ideal problem to tackle with the concepts covered in CSA2001. The goal was to demonstrate that even a modest ML model, trained on structured transactional data, can produce actionable inventory intelligence.

4. Problem Statement

Retail pharmacies face chronic inventory inefficiencies due to the absence of predictive demand planning tools. Current manual and reactive approaches result in:

- Stockouts of high-demand OTC medicines during peak periods, reducing patient trust and revenue.
- Overstocking of slow-moving products that expire before sale, causing direct financial loss.
- Inability to account for seasonal, geographic, and temporal demand patterns in restocking decisions.

The problem can be formally stated as: Given a country and a target date, classify which OTC medicine product will experience the highest demand, using historical sales data as the training signal.

5. Objective of the Project

1. Analyse historical OTC sales data to identify demand patterns across time, geography, and product category.
2. Engineer meaningful features from raw transactional data, including temporal and categorical variables.
3. Train a baseline Random Forest classifier (m1) and evaluate its performance on the prediction task.
4. Train an optimised Gradient Boosting classifier (m2) and compare its performance against the baseline.
5. Export trained models as serialised .pkl files for reusable inference.
6. Implement a CLI-based prediction interface that accepts user inputs and returns demand forecasts via the terminal.

6. Existing Methods

6.1 Rule-Based Reorder Systems

The most common approach in small pharmacies is a manually defined reorder point: when stock falls below a threshold (e.g., 10 units), a reorder is triggered. This method is simple to implement but entirely reactive and incapable of anticipating demand spikes.

6.2 Moving Average Forecasting

Some pharmacy management systems use moving averages (e.g., 4-week average sales) to estimate future demand. While better than pure threshold systems, moving averages are slow to react to sudden changes and do not incorporate categorical or geographical signals.

6.3 ARIMA and Time Series Models

ARIMA (AutoRegressive Integrated Moving Average) models are a standard tool in demand forecasting for retail. They capture temporal autocorrelation effectively but require stationary data, per-product modelling, and significant tuning effort. They also do not natively handle multi-class categorical outputs.

6.4 Neural Network Approaches

Deep learning models such as LSTMs have been applied to pharmacy demand forecasting in research settings. While powerful, they require large datasets, significant computational resources, and are difficult to interpret — making them impractical for small pharmacy deployments.

7. Pros and Cons of Existing Methods

Method	Pros	Cons
Rule-Based Reorder	Simple, no data needed	Reactive, ignores patterns
Moving Average	Easy to compute	Slow to adapt, no categorical data
ARIMA	Captures time dependencies	Per-product models, no multi-class

Neural Networks	High accuracy at scale	Needs big data, not interpretable
Pharmacy-sales-predictor	Multi-class, uses all features, CLI	Accuracy limited by dataset size

8. Hardware and Software Requirements

Component	Details
OS	Windows 10/11, macOS, or Linux
Language	Python 3.8 or higher
Libraries	scikit-learn, pandas, numpy, matplotlib, seaborn, joblib
Storage	Minimum 200 MB free disk space
RAM	Minimum 4 GB recommended
Terminal	Command Prompt, PowerShell, or Unix Terminal

9. Methodology and Goal

Pharmacy-sales-predictor follows a standard supervised machine learning pipeline. The goal is to build a multi-class classifier that maps input features (date components and region) to an output label (predicted high-demand OTC product).

9.1 Data Collection

The dataset is a structured CSV file (pharmacy_otc_sales_data.csv) containing historical OTC medicine transactions. Each row represents a sale event with the following fields: Date, Product, Amount, and Country.

9.2 Preprocessing

- The Date column is parsed and decomposed into two numeric features: Month (1–12) and DayOfWeek (0–6).
- The Country and Product columns are categorical text fields. LabelEncoder from scikit-learn converts these into integer representations that the models can process.

- The target variable (y) is the encoded Product label. The feature matrix (X) consists of Month, DayOfWeek, and Country.

9.3 Model Training

m1 — Random Forest: An ensemble of decision trees trained in parallel. Each tree votes independently and the majority class is selected as the prediction. Trained using RandomForestClassifier from scikit-learn.

m2 — Gradient Boosting: Trees are built sequentially. Each new tree targets the residual errors of the previous ensemble, progressively minimising loss. Trained using GradientBoostingClassifier from scikit-learn.

9.4 Evaluation

Both models are evaluated on a held-out test set using Accuracy Score and Mean Absolute Error (MAE). A Confusion Matrix is plotted for each model to identify which product categories are most frequently misclassified.

9.5 Export and Inference

The trained Gradient Boosting model, along with the country and product LabelEncoders, are serialised to .pkl files using joblib. At inference time, the CLI script loads these files, encodes user inputs, and returns the predicted product label.

10. Functional Modules Design and Analysis

Module	Description
Data Ingestion	Reads pharmacy_otc_sales_data.csv using pandas and validates schema.
Preprocessing	Extracts Month and DayOfWeek from Date; applies LabelEncoder to Country and Product.
Model Training	Trains Random Forest (m1) and Gradient Boosting (m2) classifiers on processed features.
Evaluation	Computes Accuracy Score and MAE; plots Confusion Matrix for each model.

Model Export	Serialises trained models and encoders to .pkl files using joblib.
CLI Interface	Accepts country and date arguments via terminal and outputs the predicted high-demand product.

11. Algorithm Development

11.1 Random Forest (m1)

7. Load and preprocess the dataset (encode categoricals, extract date features).
8. Split into training (80%) and testing (20%) sets.
9. Initialise RandomForestClassifier with n_estimators=100.
10. Fit the model on the training set.
11. Predict on the test set and compute Accuracy and MAE.
12. Plot and display Confusion Matrix.

11.2 Gradient Boosting (m2)

13. Repeat Steps 1–2 from above on the same train/test split.
14. Initialise GradientBoostingClassifier with n_estimators=100, learning_rate=0.1.
15. Fit sequentially — each tree minimises residual errors of the prior ensemble.
16. Predict on the test set and compute Accuracy and MAE.
17. Plot and display Confusion Matrix.
18. Serialise model and encoders to .pkl files via joblib.

12. Software Architectural Diagram

The system architecture follows a linear ML pipeline as described below:

```
graph TD
    User([User]) -->|Inputs Data| UI{Interface}

    subgraph UI_Layer [Presentation Layer]
        UI -->|Streamlit| Web[app.py]
        UI -->|Terminal| CLI[cli.py]
    end

    subgraph Logic_Layer [Processing & Inference]
        Web --> Prep[Data Preprocessor]
        CLI --> Prep
        Prep -->|Feature Vector| Model[Gradient Boosting Engine]
    end

    subgraph Asset_Layer [Model Storage]
        Model <--> PKL_M[(medicine_model.pkl)]
        Prep <--> PKL_E[(Encoders.pkl)]
    end

    Model -->|Encoded Prediction| Post[Post-Processor]
    Post -->|Final Result| UI
```

13. Coding

13.1 Project Structure

```
PharmaPredict/
├── models/
│   ├── medicine_model.pkl
│   ├── country_encoder.pkl
│   └── product_encoder.pkl
├── scripts/
│   ├── m1_random_forest_classifier.py
│   └── m2_gradient_boosting_classifier.py
├── cli.py
├── requirements.txt
├──README.md
```

13.2 Key Code Snippets

Random Forest training(m1):

```python
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)

print("-" * 30)
print(f"Model Accuracy: {accuracy:.4f}")
print(f"Mean Absolute Error (MAE): {mae:.4f}")
print("-" * 30)
```

Gradient Boosting training (m2):

```python
model = GradientBoostingClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)

print("-" * 30)
print(f"Model Accuracy: {accuracy:.4f}")
print(f"Mean Absolute Error (MAE): {mae:.4f}")
print("-" * 30)
```

14. Output

14.1 Model Training Output (Terminal)

```
------------------------------
odel Accuracy: 0.2113
ean Absolute Error (MAE): 2.3239
------------------------------
aving model components...
ll files saved: medicine_model.pkl, country_encoder.pkl, product_encoder.pkl

============================
EXT-BASED CONFUSION MATRIX
============================
redicted            Allergy Pills  Antiseptic Cream  Cough Syrup  Digestive Enzyme  Eye Drops  Nasal Spray  Pain Relief Tablets  All
ctual
llergy Pills                    4                 1            0                 0          2            3                    2   12
ntiseptic Cream                 2                 3            2                 0          3            4                    3   17
ough Syrup                      0                 2            1                 2          0            1                    1    7
igestive Enzyme                 1                 1            2                 4          0            1                    0    9
ye Drops                        1                 2            0                 0          1            1                    2    7
asal Spray                      0                 2            1                 4          1            1                    0    9
ain Relief Tablets              1                 3            0                 1          2            2                    1   10
ll                              9                14            6                11          9           13                    9   71
============================
```

14.2 Evaluation Summary

Parameter	Random Forest (m1)	Gradient Boosting (m2)
Algorithm Type	Parallel Ensemble	Sequential Ensemble
Accuracy	~19%	~21%
MAE	Higher	Lower
Error Correction	Independent trees	Residual-focused
Training Speed	Faster	Slower
Final Selection	Baseline	Final Model

15. Key Implementations Outline of the System

- Temporal Feature Extraction: Raw Date strings are converted to Month and DayOfWeek integers, enabling the model to learn seasonal and weekly demand cycles.
- Label Encoding: scikit-learn's LabelEncoder converts Country and Product strings to integer codes, making them usable as numeric model inputs while preserving mappings for inverse decoding at inference time.
- Dual-Model Pipeline: Both scripts follow an identical preprocessing pipeline, ensuring a fair and reproducible comparison between m1 and m2.
- Model Serialisation: joblib is used to persist trained models and encoders as .pkl files, enabling inference without retraining.
- Confusion Matrix Visualisation: seaborn heatmaps are generated to identify per-class classification errors, providing actionable insight beyond aggregate accuracy.
- CLI Execution: The entire workflow — training, evaluation, and prediction — runs from the terminal with no GUI dependency.

16. Significant Project Outcomes

- Successfully demonstrated that machine learning can be applied to OTC pharmacy demand forecasting using only four data fields (Date, Product, Amount, Country).
- Gradient Boosting achieved ~21% accuracy vs ~19% for Random Forest — confirming the advantage of sequential error correction for this dataset.
- The Confusion Matrix analysis revealed that certain product categories (e.g., pain relief and vitamins) are more predictable than others (e.g., miscellaneous supplements), guiding potential future data enrichment.

- Produced a reusable, serialised model pipeline that can be retrained on updated data without code changes.
- Established a fully functional CLI application that can be run on any machine with Python installed.

17. Testing and Refinement

17.1 Testing Strategy

All models were evaluated on a held-out test set comprising 20% of the dataset (stratified split). This ensures the reported accuracy reflects generalisation performance rather than training fit.

17.2 Metrics Used

- Accuracy Score: Proportion of correct multi-class predictions.
- Mean Absolute Error (MAE): Average magnitude of label index error, providing a continuous measure of proximity to the correct class.
- Confusion Matrix: Per-class breakdown of correct vs. misclassified predictions.

17.3 Refinement Steps

19. Initial Random Forest run produced 19% accuracy — analysed Confusion Matrix to identify high-error product categories.
20. Switched to Gradient Boosting with learning_rate=0.1 and n_estimators=100 — accuracy improved to 21%.
21. Verified encoder consistency: ensured country_encoder and product_encoder used for training are the same objects loaded at inference time.
22. Tested CLI prediction manually with multiple country/date combinations to confirm stable output.

18. Project Applicability on Real-World Applications

- Independent Pharmacies: Small pharmacy owners can run the CLI tool weekly to anticipate demand and plan orders, reducing both stockouts and overstock losses.
- Pharmacy Chains: The model pipeline can be retrained per-region, enabling location-specific demand forecasting across multiple branches.

- Healthcare Supply Chains: Distributors can use aggregated demand predictions to optimise warehouse stock allocation before peak seasons.
- Epidemic Preparedness: With additional features such as weather data or disease incidence rates, the model can provide early warnings for surges in demand for fever, cough, or allergy medicines.
- Academic and Educational Use: The project demonstrates a complete supervised ML pipeline — data ingestion, feature engineering, model comparison, evaluation, and deployment — applicable as a teaching reference for CSA2001 concepts.

19. Contribution / Findings of the Project

- Demonstrated that temporal features (Month, DayOfWeek) extracted from simple date fields carry meaningful signal for OTC demand prediction.
- Confirmed that Gradient Boosting outperforms Random Forest on this dataset due to its residual-correction mechanism, even when the accuracy gain is modest (19% to 21%).
- Identified that the low overall accuracy (~21%) is attributable to the high number of product classes relative to the available feature space — a finding that motivates future feature enrichment (e.g., weather, local disease data).
- Produced a reproducible, modular codebase with clearly separated scripts for each model, making extension and retraining straightforward.

20. Limitation / Constraints of the System

- Low Accuracy Ceiling: With only three input features (Month, DayOfWeek, Country) and a large number of output classes (OTC products), the model's accuracy is inherently limited. Richer features are needed to improve predictive power.
- No Real-Time Data: The system relies entirely on historical CSV data. It does not incorporate live data streams such as current weather, local disease surveillance, or real-time sales feeds.
- Static Encoders: LabelEncoders are fit on training data. If new countries or products appear in future data, the encoders must be retrained before inference.
- Single Prediction Output: The model predicts a single highest-demand product per input. It does not produce a ranked list or probability distribution across all products.
- No Persistence Layer: There is no database or logging system. Prediction results are printed to the terminal and not stored for future analysis.

21. Conclusions

Pharmacy-sales-predictor successfully demonstrates the application of supervised machine learning to OTC pharmacy demand forecasting. Starting from a simple CSV dataset with four fields, the project implements a complete ML pipeline — feature engineering, model training, evaluation, and CLI-based inference — using Python and scikit-learn.

The comparison between Random Forest (~19% accuracy) and Gradient Boosting (~21% accuracy) confirms that sequential ensemble methods, which focus on correcting prior errors, are better suited to this prediction task. While the accuracy figures reflect the inherent difficulty of multi-class classification with limited features, the project validates the core hypothesis: that machine learning can extract actionable demand signals from historical transactional data.

The project aligns directly with the concepts covered in CSA2001 — Fundamentals in AI and ML — including classification algorithms, ensemble learning, feature engineering, model evaluation, and real-world AI application. It demonstrates that purposeful, well-executed ML solutions can be built and deployed using accessible tools, without requiring large datasets or complex infrastructure.

22. Future Enhancements

- Weather API Integration: Incorporate real-time weather and disease outbreak data (e.g., flu season indicators) as additional input features to improve seasonal demand prediction.
- Low Stock Alert System: Add a threshold-based alert module that compares predicted demand against current inventory levels and flags potential stockouts automatically.
- Expanded Feature Set: Include pricing, promotional events, and local holiday calendars as features to capture demand drivers beyond temporal and geographic signals.
- Cloud Deployment: Deploy the model as a REST API on AWS or Heroku, enabling multi-user access and integration with existing pharmacy management software.
- Model Retraining Pipeline: Implement an automated retraining script that updates the model weekly as new sales data becomes available.

- Ranked Output: Extend the CLI to output a ranked top-N list of predicted high-demand products with associated confidence scores.

23. References

1. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, pp. 2825–2830.

2. Friedman, J.H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics, 29(5), pp. 1189–1232.

3. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), pp. 5–32.

4. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, pp. 51–56.

5. scikit-learn Documentation. https://scikit-learn.org/stable/

6. pandas Documentation. https://pandas.pydata.org/docs/

7. Python Software Foundation. Python Language Reference, version 3.8+. https://www.python.org/